Introduction to Big Data Processing

In this train, we'll discover the expansive world of Big Data, designed to unveil the vast amounts of data that surround us and the necessity of specialized processing methods. We'll dive into the intricacies of handling, analyzing, and deriving insights from large datasets, and build a foundational understanding of big data, its challenges, and the platforms where data processing takes place.

Learning Objectives

- Know why we have to process big data differently.
- Understand the different components of Hadoop architecture.
- Know which big data processing tools exist.
- Know where we will be doing our processing.

Why Process Big Data Differently?

The landscape of data has evolved dramatically over the past few decades. From the introduction of relational databases in the 1970s to the formalization of the data engineering role in 2015, the volume and complexity of data have grown exponentially. In 2013, IBM stated that 90% of all data had been created in the last two years , and by 2020, big data was growing at a rate of 40% per year. This rapid growth is expected to continue, driven by the increasing number of internet-connected devices.

This "flood of data" means we need to process data more efficiently. However, due to physical limitations, compute speeds are not improving at the rate predicted by Moore's Law. This creates a significant challenge: how do we handle ever-increasing data volumes with comparatively slower improvements in computational capability?

Historically, programs were written to execute serially, meaning each instruction was processed one after another on a single computer. While still true for some programming languages like Python if parallelism isn't explicitly implemented, this serial

approach creates a large bottleneck for the amount of data a single program can process.

The answer lies in **Massively Parallel Processing (MPP)** . In contrast to serial computing, distributed computation leverages multiple processing units to perform various tasks in parallel. Programs are broken into independent parts that can be executed in isolation and reassembled later. This approach can run on diverse architectures, including single CPU machines with multiple cores, large servers, or graphical processing units (GPUs) containing thousands of cores. MPP represents the extreme of this, where hundreds or thousands of computing units (nodes) work in parallel.

MPP is particularly applicable to big datasets because each dataset can be split into smaller, independent subsets for processing. This fundamental shift enables processing massive amounts of data at much quicker rates by distributing instructions to nodes in a cluster to work in parallel. This approach takes advantage of data locality, where nodes manipulate only the data sent to them, which is faster and more efficient than conventional architectures.

> **Reflection:** Massively parallel processing (MPP) is essential for big data, as compute speeds cannot keep pace with data growth. MPP enables efficient handling of large datasets by distributing tasks across multiple processors.

Apache Hadoop and Its Architecture

Apache Hadoop is a collection of open-source software specifically designed for big data processing. It leverages the parallelizable nature of massive datasets, using networks of connected computers (clusters) to process vast amounts of data.

Why Use Hadoop?

Hadoop offers several key advantages for big data processing:

- Can store and process a very large amount of data.
- Can store a variety of data – both structured and unstructured.
- It is fault-tolerant.
- It is scalable.
- Supports SQL tasks (through Hive).

Why Learn About Hadoop?

Hadoop is considered the ancestor of all big data processing, originating in 2006 based on an idea published by Google. Although its popularity has seen some decline with the emergence of other big data offerings like Apache Spark , most new big data platforms are built on top of some component of the Hadoop system. Therefore, understanding how Hadoop works is crucial for any data professional.

Hadoop Components

Apache Hadoop primarily consists of three core components:

- The **MapReduce** programming model for processing.
- The **YARN** (Yet Another Resource Negotiator) resource manager.
- A storage model known as the **Hadoop Distributed File System (HDFS)** .

The Hadoop ecosystem is extensive, supporting and forming the base for various other components:

- **Data Collection:** Flume, Kafka, Sqoop
- **Other Processing Engines:** Spark
- **Coordination and Management Extensions:** Apache Ambari, Zookeeper, Oozie, HCatalog, Pig, HBase, Cassandra, Mahout
- **SQL Query:** Hive

Hadoop Distributed File System (hdfs)

HDFS is the primary data storage system used by Hadoop applications, based on a Distributed File System (DFS).

In its simplest form, HDFS involves a master node called the **NameNode** and worker nodes called **DataNodes** .

- The **NameNode** is the master daemon. It records metadata and maintains the DataNodes.
- **DataNodes** act as workers within the cluster, storing the actual data and serving read and write requests from clients.
- The **Client** interfaces with HDFS to retrieve and send data to the cluster, handling both data and metadata.

Daemons are background processes not created through direct user interaction; in Hadoop, they are created at launch.

When a file is loaded into HDFS, it is split into blocks (default 128 MB, but configurable). Each block is then stored in one of the DataNodes on the cluster. The NameNode keeps track of the metadata, indicating where each block of data is located. The beauty of Hadoop is that all this happens in the background, abstracting the complexity from the user.

Hadoop handles failures and increased load by replicating each data block three times across different DataNodes. This fault tolerance ensures that if a node crashes, the data can be recovered from a replica, allowing processing to continue. Horizontal scaling also allows Hadoop to infinitely expand the storage capacity of the cluster.

Yet Another Resource Negotiator (yarn)

YARN plays a crucial role in Hadoop by assisting with job scheduling for various applications and managing resources within the cluster.

Key reasons why YARN works effectively:

- **Efficiency:** It separates the resource management layer from the processing layer, significantly increasing system efficiency. Progressive resource allocation ensures optimal utilization of available resources.
- **Multi-tenancy:** YARN enables different workloads (such as batch, stream, graph, and interactive) to run on Hadoop, moving beyond the single, batch-focused MapReduce processing that characterized early Hadoop deployments.
- **Scalability:** Like HDFS, YARN allows a very large number of nodes to be theoretically added to the cluster, increasing the Hadoop cluster's processing capacity.

YARN's architecture consists of several key components:

- **ResourceManager:** This is the master daemon, consisting of a Scheduler and an ApplicationsManager. It accepts and schedules jobs, allocates resources, and optimizes resource allocation. The Scheduler allocates resources based on availability , while the ApplicationsManager accepts jobs and negotiates the first container for executing the application's ApplicationsMaster.
- **NodeManager:** This acts as a monitoring and reporting agent for the ResourceManager. It manages individual node containers and their resources, and creates container processes when requested by the ApplicationMaster.

- **ApplicationsMaster:** This component monitors and tracks individual applications (single jobs). It is responsible for negotiating resources from the Scheduler and requesting containers from the NodeManager by sending a Container Launch Context (CLC).
- **Containers:** These represent physical resources (like CPU, memory, and storage) on a single node. They are launched by the CLC, which includes metadata like environment variables, security tokens, and dependencies.

Mapreduce

MapReduce is the fundamental innovation of Hadoop, designed to process large, unstructured datasets using a distributed algorithm on a cluster. The process involves four main steps:

- **Splitting:** Data is divided into fixed-size splits.
- **Mapping:** Data in each split is passed to a mapping function.
- **Shuffling:** Relevant records from the Mapping phase output are consolidated.
- **Reducing:** Output values from the Shuffling phase are aggregated.

> **Reflection:** The NameNode in HDFS acts as the master service, maintaining metadata about the data blocks and managing the DataNodes, which store the actual data.

Open-source Big Data Processing Tools

There is a massive list of applications being developed for big data processing in the open-source space, particularly as part of the Apache Software Foundation. These technologies leverage the increase in computational power and memory to achieve the most efficient way of transforming large amounts of data. This section will highlight six key emerging tools for big data processing, warehousing, or machine learning.

Massively parallel processing (MPP) techniques have been developed to assist in big data processing. This includes well-known frameworks like Apache Hadoop, Apache Spark, and Apache Hive.

Apache Storm

Apache Storm is an open-source distributed real-time computation system. It aims to do for real-time processing what Hadoop did for batch processing. Its applications include realtime analytics, online machine learning, continuous computation, ETL, and distributed remote procedure call (RPC). Apache Storm can integrate with any queueing system and any database system, for example, Apache Kafka.

When programming in Apache Storm, you manipulate and transform streams of tuples (named lists of values). It has three core abstractions:

- A **spout** is a source of streams in a computation, typically reading from a queueing broker; they are input nodes.
- A **bolt** processes input streams and produces new output streams. Most computation logic (e.g., functions, filters, joins, aggregations, database interactions) goes into bolts.
- A **topology** is a network of spouts and bolts, where each edge represents a bolt subscribing to the output stream of another spout or bolt.

Apache Hive

Apache Hive is a data warehouse project built on top of Apache Hadoop, providing a simple interface for interacting with data stored in HDFS. Hive offers a SQL-like interface (HiveQL) to integrate SQL queries with the underlying Java on which Hadoop is built. HiveQL automatically translates SQL-like queries into batch MapReduce jobs. Contributors to the Hive project include Facebook and Netflix.

Hive's architecture includes a **Metastore**, which is the central repository for table metadata (not the actual data). This metadata includes location, schema, partitioning information, and other important data. The Metastore tracks and replicates data, providing a backup in case of data loss. Apache Hive Metastore can be built on top of other big data offerings like Hadoop and Spark, enabling SQL querying of distributed data.

The **Driver** acts as a controller, receiving statements, monitoring execution progress, and creating sessions. It also stores metadata created during HiveQL execution. The **Compiler** converts HiveQL queries into MapReduce input. The **Optimiser** performs transformations for aggregation and pipeline conversion, and splits tasks for efficiency and scalability. The **Executor** executes tasks after compilation and optimization, interacting directly with the Hadoop Job Tracker for scheduling. Users interact with Hive via the Command Line Interface (CLI), User Interface (UI), and Thrift Server.

Apache Spark

Apache Spark is a unified analytics engine for big data processing, providing high-level APIs in Python, Scala, and Java. Spark allows processing of batch, streams, and graphs, and performing machine learning through various APIs. It follows a simple architecture focused solely on computation implementation.

The **Master Node** contains a **driver program** that coordinates task execution. Within the driver program, a Spark Context is defined where all execution occurs. Cluster resources are maintained by a **Cluster Manager** , which manages **Worker Nodes** where tasks are executed through an **Executor** .

Apache Flink

Apache Flink is an open-source, unified stream and batch data processing framework. Its core is a dataflow engine written in Java and Scala. It executes arbitrary dataflow programs in a data-parallel and pipelined manner for both batch and streaming. The dataflow programming model provides event-at-a-time processing on both finite and infinite datasets. Flink offers high levels of fault tolerance and exactly-once semantics for streaming workloads. It supports APIs in Java, Scala, Python, and SQL.

Flink has two core APIs: a DataStream API for bounded and unbounded data streams, and a DataSet API for bounded datasets. It also supports a Table API for relational processing via SQL, which can be embedded into the DataStream and DataSet APIs. Flink programs are mapped into dataflows that start with sources and include transformations, represented as directed acyclic dataflow graphs, allowing branching and merging of dataflows.

Apache Samza

Apache Samza is an open-source, near real-time, asynchronous framework for stream processing, developed in Scala and Java. Samza allows users to build stateful applications that process data in near real-time for systems like Apache Kafka and other sources. Processing data in streams enables sub-second response times. A stream in Samza consists of a collection of immutable messages, typically of the same type and modeled as key-value pairs. Streams can have multiple producers or consumers and can be unbounded (e.g., Apache Kafka) or bounded (e.g., HDFS). Like Flink, stream applications process messages from input streams, transform them, and emit results to an output stream or database. It is used by companies like LinkedIn, Slack, and TripAdvisor.

Apache Samoa

Apache Samoa is a distributed streaming machine learning (ML) framework, currently in incubation, that provides programming abstractions for distributed streaming ML algorithms. Essentially, Samoa leverages the computational advantages of Apache Storm, Apache Samza, and Apache Flink to provide an abstracted view for building machine learning models on data streams. It offers a collection of distributed streaming algorithms for common data mining and machine learning tasks such as classification, clustering, regressions, and other operational implementations required for building more complex algorithms.

> **Reflection:**
>
> Apache Hive provides a SQL-like interface for querying data stored in HDFS, making it easier for users to interact with big data using familiar query syntax.

Where We Will Be Doing Our Processing

All the open-source solutions discussed above are software tools that still require hardware to run on. Apache itself is not an infrastructure provider. You have a few options for deploying and running these solutions:

1. **Buy infrastructure and host the software and servers yourself:** This involves setting up and managing your own physical hardware.
2. **Go with an enterprise-grade system:** These are commercial offerings that provide robust hosted solutions, often with on-demand support and tailor-made setups. An example is Cloudera, which offers a software platform for data engineering, warehousing, machine learning, and analytics. Cloudera's initial implementation was an optimized version of Apache Hadoop for enterprise deployments. Many other commercial providers focus on enterprise or big data solutions.
3. **Choose from a range of technology providers:** These providers offer either infrastructure to host services on or already-hosted platforms.

The choice of provider will largely depend on the specific needs of your organization. It's important to remember that for generic applications, there are many major players offering solutions for data storage, processing, and presentation.

> **Reflection:**
>
> Big data processing software like Apache Spark can be deployed using either enterprise-grade hosted solutions or through infrastructure and platform providers offering flexible cloud-based services.

Conclusion

In this train, we've covered a wide array of technologies, solutions, and implementations available to a data engineer for big data processing. While this may seem like a lot to absorb, the key takeaway is not to get bogged down by specific toolsets, but rather to focus on the problem you are trying to solve. It's crucial to understand that there are many ways to approach big data problems and many toolkits to achieve your goals.

Key Takeaways

- **Why Big Data Processing:** Exponential data growth outpaces computational speed improvements, necessitating massively parallel processing (MPP).
- **Hadoop Architecture:** Comprised of HDFS (distributed storage), YARN (resource management), and MapReduce (distributed processing), forming the foundation for many big data tools.
- **Open-Source Tools:** A rich ecosystem of Apache tools exists for various big data tasks, including Storm (real-time), Hive (data warehousing), Spark (general processing), Flink (unified stream/batch), Samza (stream processing), and Samoa (streaming ML).
- **Deployment Options:** Big data software can be run on self-managed infrastructure, enterprise-grade systems, or platforms offered by technology providers.
- **Focus on Problem:** The most important aspect is understanding the problem and choosing appropriate principles and tools, rather than memorizing specific toolsets.